

Mini-rapport – Coloration de grands graphes réels

KARA LYDIA

KARAOULI MERIAM

4 mai 2026

1 Introduction

La coloration de graphes est un problème classique de théorie des graphes. Elle consiste à attribuer une couleur à chaque sommet d'un graphe de façon à ce que deux sommets voisins aient toujours des couleurs différentes. Une telle coloration est dite propre. Le plus petit nombre de couleurs permettant de colorier correctement un graphe G est appelé nombre chromatique et est noté $\chi(G)$.

Ce problème apparaît dans de nombreuses applications concrètes. Par exemple, il peut modéliser des problèmes d'ordonnancement, d'allocation de ressources, de planification d'examens, d'affectation de fréquences dans un réseau ou encore d'allocation de registres en compilation. Dans tous ces cas, deux objets incompatibles sont reliés par une arête, et les couleurs représentent des ressources ou des créneaux différents.

Cependant, déterminer exactement le nombre chromatique d'un graphe quelconque est un problème difficile. Les algorithmes exacts deviennent rapidement inutilisables lorsque le nombre de sommets augmente. Pour cette raison, on utilise souvent des algorithmes heuristiques, qui ne garantissent pas toujours une solution optimale, mais qui donnent une coloration correcte en temps raisonnable.

Dans ce projet, on s'intéresse principalement à l'algorithme DSATUR, étudié en cours. Pour un graphe G d'ordre n , on note $D(G)$ le nombre de couleurs utilisées par DSATUR. Même si DSATUR ne donne pas toujours le nombre chromatique exact, on a toujours :

$$\chi(G) \leq D(G).$$

Ainsi, DSATUR fournit une majoration du nombre chromatique.

L'objectif du projet est d'étudier le comportement de DSATUR sur de grands graphes réels. Plus précisément, on observe à quel moment l'algorithme utilise sa dernière couleur. Si $n_i(G)$ désigne le nombre de sommets déjà coloriés lorsque la couleur i apparaît pour la première fois, alors on étudie le ratio :

$$\rho = \frac{n_k(G)}{n}, \quad \text{avec } k = D(G).$$

Ce ratio permet de savoir si la dernière couleur apparaît tôt ou tard dans l'exécution de l'algorithme. Un ratio faible signifie que DSATUR utilise très vite toutes ses couleurs. Un ratio proche de 1 signifie que la dernière couleur apparaît seulement vers la fin de la coloration.

2 Représentation du graphe

Le premier travail réalisé a été de choisir une structure de données adaptée au projet. En cours, deux représentations principales ont été étudiées : la matrice d'adjacence et les listes d'adjacence.

La matrice d'adjacence est simple à utiliser. Pour un graphe à n sommets, elle consiste à créer un tableau de taille $n \times n$. La case (i, j) indique si les sommets i et j sont voisins. Cette représentation permet de tester très rapidement l'existence d'une arête, en temps constant. En revanche, elle consomme beaucoup de mémoire lorsque le graphe est grand.

Dans ce projet, les graphes étudiés peuvent contenir un grand nombre de sommets. De plus, les graphes réels sont souvent creux, c'est-à-dire que le nombre d'arêtes est faible par rapport au nombre maximal possible d'arêtes. Pour cette raison, la représentation par listes d'adjacence est plus adaptée. Chaque sommet possède une liste contenant ses voisins.

Le graphe est donc codé en C++ par :

```
vector<vector<int> > voisin;
```

où `voisin[i]` contient la liste des voisins du sommet `i`. Cette structure est proche de celle utilisée dans les TP, notamment dans le TP sur Dijkstra où les voisins étaient stockés dans un tableau de `vector<int>`.

Les sommets sont renumérotés en interne de 0 à $n - 1$. Ce choix simplifie l'accès aux tableaux de couleurs, de degrés et de saturation. Les noms initiaux des sommets sont tout de même conservés afin de pouvoir afficher les résultats avec les identifiants d'origine.

3 Lecture des fichiers de graphes

Le programme permet de lire des graphes provenant de plusieurs formats simples. Le premier format pris en charge est le format DIMACS, souvent utilisé pour les instances classiques de coloration. Dans ce format, une ligne du type

```
p edge n m
```

indique le nombre de sommets et d'arêtes, puis chaque arête est donnée par une ligne :

```
e u v.
```

Le programme accepte aussi les formats de type SNAP ou KONECT, qui utilisent généralement une liste d'arêtes. Dans ce cas, chaque ligne contient simplement deux sommets :

```
u v.
```

Les lignes de commentaires sont ignorées. Le programme ignore également les boucles, car une boucle rendrait impossible une coloration propre classique. Les arêtes multiples sont supprimées après la lecture du fichier.

Cette étape est importante, car les graphes réels peuvent contenir des petites variations de format : sommets numérotés à partir de 0 ou de 1, commentaires, ou encore doublons. Le programme essaye donc de rester robuste tout en gardant une implémentation simple.

4 Algorithmes implémentés

Trois algorithmes de coloration ont été implémentés.

4.1 Glouton dans l'ordre naturel

Le premier algorithme est l'algorithme glouton classique. Les sommets sont coloriés dans l'ordre naturel, c'est-à-dire dans l'ordre dans lequel ils sont stockés après la lecture. Pour chaque sommet, on choisit la plus petite couleur qui n'est pas utilisée par ses voisins déjà coloriés.

Cet algorithme est très simple et rapide. Cependant, il dépend fortement de l'ordre des sommets. Deux ordres différents peuvent donner des nombres de couleurs très différents.

4.2 Glouton par degrés décroissants

Le deuxième algorithme est une variante du glouton. Avant la coloration, les sommets sont triés par degré décroissant. L'idée est de commencer par les sommets les plus contraints, c'est-à-dire ceux qui ont le plus de voisins.

Cette idée est proche de l'algorithme de Welsh et Powell vu en cours. Elle permet souvent d'améliorer la coloration par rapport à l'ordre naturel, mais elle reste une heuristique. Elle ne garantit pas toujours un résultat optimal.

4.3 DSATUR

Le troisième algorithme est DSATUR, qui est l'algorithme principal du projet. Contrairement au glouton classique, DSATUR ne fixe pas un ordre définitif des sommets au début. L'ordre est recalculé à chaque étape.

Pour chaque sommet non colorié, on calcule son degré de saturation. Celui-ci correspond au nombre de couleurs différentes déjà présentes dans son voisinage. À chaque étape, DSATUR choisit un sommet non colorié ayant un degré de saturation maximum. En cas d'égalité, on choisit celui de plus grand degré. Dans notre implémentation, s'il y a encore égalité, on choisit celui de plus petit indice.

Une fois le sommet choisi, il est colorié avec la plus petite couleur disponible. Les degrés de saturation de ses voisins sont ensuite mis à jour.

Le choix de DSATUR est intéressant car il prend en compte les contraintes qui apparaissent progressivement pendant la coloration. En pratique, il donne souvent de meilleures colorations que le glouton simple. En revanche, il est plus coûteux en temps de calcul, surtout sur de très grands graphes.

5 Calcul du ratio étudié

En plus du nombre de couleurs utilisées, le programme calcule les valeurs $n_i(G)$. La valeur $n_i(G)$ correspond au nombre de sommets déjà coloriés lorsque la couleur i apparaît pour la première fois.

Par exemple, on a toujours :

$$n_1(G) = 1$$

pour un graphe non vide, car le premier sommet colorié reçoit la couleur 1. De même, si le graphe contient au moins une arête, la couleur 2 apparaît généralement très tôt.

Si DSATUR utilise k couleurs, alors le ratio étudié est :

$$\rho = \frac{n_k(G)}{n}.$$

Ce ratio donne une information différente du simple nombre de couleurs. Deux graphes peuvent être coloriés avec le même nombre de couleurs, mais la dernière couleur peut apparaître très tôt dans un cas et très tard dans l'autre.

Un ratio faible peut indiquer que la difficulté de coloration apparaît rapidement. Un ratio élevé peut indiquer que l'algorithme parvient longtemps à colorier les sommets avec moins de couleurs, puis doit ajouter une nouvelle couleur seulement vers la fin.

6 Résultats obtenus

Les premiers tests ont été réalisés sur de petits graphes afin de vérifier le fonctionnement du programme. Le tableau suivant présente les résultats obtenus sur un graphe d'exemple.

Graphe	n	m	Algorithme	Couleurs	Ratio ρ
exemple_aretes	6	8	Glouton naturel	3	0.5000
exemple_aretes	6	8	Glouton degrés	3	0.8333
exemple_aretes	6	8	DSATUR	3	0.5000

TABLE 1 – Résultats sur un petit graphe de test.

Sur ce graphe, les trois algorithmes utilisent trois couleurs. Cependant, le ratio n'est pas le même pour tous les algorithmes. Pour le glouton naturel et DSATUR, la troisième couleur apparaît après avoir colorié trois sommets, ce qui donne un ratio de 0.5. Pour le glouton par degrés décroissants, la troisième couleur apparaît plus tard, avec un ratio de 0.8333.

Ce résultat montre que le ratio apporte une information supplémentaire. Le nombre de couleurs seul ne suffit pas à décrire complètement le comportement de l'algorithme.

Les tableaux suivants sont prévus pour comparer ensuite plusieurs graphes réels issus de DIMACS, SNAP ou KONECT.

Graphe	n	m	Couleurs DSATUR	n_k	Ratio ρ
soc-Epinions1	75879	405740	30	83	0.0011
Wiki-Vote	7115	100762	24	279	0.0392
wiki-RfA	10603	177875	30	155	0.0146

TABLE 2 – Résultats obtenus avec DSATUR sur les graphes réels.

Graphe	Glouton naturel	Glouton degrés	DSATUR	Meilleur résultat
soc-Epinions1	42	41	30	DSATUR
Wiki-Vote	37	28	24	DSATUR
wiki-RfA	44	35	30	DSATUR

TABLE 3 – Comparaison du nombre de couleurs selon l'algorithme.

7 Limites observées

La première limite concerne les algorithmes exacts. L'algorithme exact vu en TP repose sur une recherche exhaustive par récursion. Il teste toutes les possibilités de coloration jusqu'à trouver une coloration valide ou prouver qu'il n'en existe pas avec k couleurs. Cette méthode est correcte, mais elle devient très lente dès que le nombre de sommets augmente. Pour cette raison, elle n'a pas été utilisée pour les grands graphes.

La deuxième limite concerne DSATUR. Dans notre implémentation, à chaque étape, le programme parcourt tous les sommets non coloriés pour choisir celui qui a le plus grand degré de saturation. Cette méthode est simple et conforme à l'algorithme vu en cours, mais elle peut devenir coûteuse pour de très gros graphes.

Une amélioration possible serait d'utiliser une structure permettant de retrouver plus rapidement le sommet de saturation maximale. Cependant, cela rendrait le code plus complexe. Pour ce projet, le choix a été de privilégier une version claire, facilement vérifiable et proche du cours.

Une autre limite vient de la lecture des graphes réels. Les formats DIMACS, SNAP et KONECT sont proches, mais pas toujours identiques. Certains fichiers peuvent contenir des sommets non consécutifs, des commentaires, des poids ou des informations supplémentaires. Le programme gère les cas simples, mais il peut être nécessaire d'adapter légèrement la lecture pour certains jeux de données particuliers.

8 Conclusion

Ce projet a permis de mettre en pratique plusieurs notions vues en cours de graphes : représentation par listes d'adjacence, parcours des voisins, degrés, coloration gloutonne et algorithme DSATUR. Le programme obtenu permet de lire un graphe réel, de le colorier par plusieurs heuristiques et de calculer le ratio demandé dans le sujet.

Les premiers résultats montrent que le nombre de couleurs ne suffit pas toujours pour comparer les algorithmes. Le ratio ρ permet d'observer plus finement le moment où la dernière couleur apparaît. Cette information est utile pour comprendre le comportement de DSATUR sur des graphes de structures différentes.

Le travail pourrait être poursuivi en testant davantage de graphes réels, puis en comparant les ratios selon les familles de graphes : réseaux sociaux, graphes biologiques, graphes d'infrastructure ou instances DIMACS difficiles.

9 Sources utilisées

Les documents utilisés sont principalement les supports fournis dans le cadre du cours :

- CM Graphes Chapitre 1 : définitions, chemins, cycles, connexité, représentations des graphes ;
- CM Graphes Chapitre 2 : parcours, arbres, plus courts chemins et flots ;

- CM Graphes Chapitre 3 : coloration de graphes, glouton, DSATUR, complexité et algorithmes exacts ;
- TP1 : graphes aléatoires et composantes connexes ;
- TP2 : listes de voisins et plus courts chemins ;
- TP3 : coloration exacte, glouton et DSATUR ;
- sujet du projet : définition de $n_i(G)$, du ratio n_k/n et objectifs expérimentaux.

Une IA générative de type ChatGPT a été utilisée comme aide pour structurer le projet, expliquer les algorithmes.